

Jayashree Nair

A Neural Framework for Sandhi Splitting in Low-Resource Sanskrit Texts



Project



AIE Project



Amrita Vishwa Vidyapeetham

Document Details

Submission ID

trn:oid::1:3577038393

Submission Date

May 23, 2026, 2:34 PM GMT+5:30

Download Date

May 23, 2026, 2:38 PM GMT+5:30

File Name

S8_Project_Report.pdf

File Size

1.2 MB

51 Pages

12,232 Words

73,177 Characters

27% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.



48 AI-generated only 27%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

A Neural Framework for Sandhi Splitting in Low-Resource Sanskrit Texts

A PROJECT REPORT

Submitted in partial fulfilment for the Degree of
Bachelor of Technology in
Computer Science and Engineering
(Artificial Intelligence)
under the School of Computing

Submitted by

Register No	Names of Students
AM.EN.U4AIE22002	Abhiram A S
AM.EN.U4AIE22028	Karthik Narayan C
AM.EN.U4AIE22119	Harigovind C B



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMRITA VISHWA VIDYAPEETHAM

Amritapuri Campus (INDIA)

May 2026

Department of Computer Science and Engineering
AMRITA VISHWA VIDYAPEETHAM
AMRITAPURI CAMPUS



BONAFIDE CERTIFICATE

This is to certify that this is a bonafide record of the project presented by the students whose names are given below during Project Phase II in partial fulfilment of the requirements of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence).

Roll No	Names of Students
---------	-------------------

AM.EN.U4AIE22002	Abhiram A S
AM.EN.U4AIE22028	Karthik Narayan C
AM.EN.U4AIE22119	Harigovind C B

Project Guide

Ms. Jayashree Nair

Project Coordinator

Dr. Veena G

Chairperson

Dept. of CSE(AI)

Reviewer

Place : Amritapuri

Date : 25 May 2026

Department of Computer Science and Engineering
AMRITA VISHWA VIDYAPEETHAM
AMRITAPURI CAMPUS



DECLARATION

We, Abhiram A S (AM.EN.U4AIE22002), Karthik Narayan C (AM.EN.U4AIE22028), and Harigovind C B (AM.EN.U4AIE22119) hereby declare that this project entitled “**A Neural Framework for Sandhi Splitting in Low-Resource Sanskrit Texts**” is a record of the original work done by us under the guidance of Ms Jayashree, Dept. of Computer Science and Engineering, Amrita Vishwa Vidyapeetham, that this work has not formed the basis for any degree/diploma/associationship/fellowship or similar awards to any candidate in any university to the best of our knowledge.

Place : Amritapuri

Date : 25th May 2026

Signature of the Project Guide

Contents

Acknowledgements	iii
List of figures	iv
List of tables	v
Abstract	vi
1 Introduction	1
1.1 Background	1
1.2 Problem Addressed	2
1.3 Motivation	3
1.4 Scope of the Project	3
2 Literature Review	5
2.1 Existing System Study	5
2.1.1 Review of Existing Research Work	5
2.1.2 Review of Existing Tools and Products	6
2.2 Research Gap	8
2.3 Problem Statement and Contributions	10
3 Proposed Work	11
3.1 Proposed Work	11
3.1.1 Objectives of the Proposed Work	12
3.2 Methodology	12
3.2.1 Overview of the Approach	12
3.2.2 Dataset Selection	13
3.2.3 Model Design	14
3.2.4 Tools and Technologies	15
3.2.5 Algorithm or Model Description	15
3.2.6 Expected Outcomes	17
3.2.7 Advantages of the Proposed Work	18
3.2.8 Limitations and Assumptions	19

4	Experimentation and Result Analysis	20
4.1	Experimental Setup	20
4.2	Datasets	21
4.3	Evaluation Metrics	21
4.4	Experimental Design	22
4.4.1	Experimental Scenarios	23
4.4.2	Parameter Tuning	23
4.5	Results	24
4.6	Analysis of Results	27
4.7	Observations	28
4.8	Comparative Analysis	28
5	Conclusion and Scope for Further Research	30
5.1	Summary of the Work	30
5.2	Research Contributions	31
5.3	Conclusion	31
5.4	Scope for Further Research	32
A	Additional Experimental Observations	34
A.1	Model Performance Summary	34
B	Analysis of Errors and Limitations	36
C	Implementation Details	37
D	Future Scope	38
	Alignment with United Nations Sustainable Development Goals (SDGs)	39
	References	41

Acknowledgements

We would like to thank our Project Guide, **Ms. Jayashree Nair**, for her continuous support, guidance valuable advice, and scrupulous focus on every detail of our project named “**A Neural Framework for Sandhi Splitting in Low-Resource Sanskrit Texts**”. Without her special skills and unwavering motivation, it would be difficult to achieve the high quality and lead this research work to the present level. Besides, we express our gratitude to our Coordinator, **Dr. Veena G**, for her persistent work and commitment to escalating and facilitating the project processes. Besides these, we would like to thank **Dr. Geetha M**, the Chairperson of the Department of Computer Science and Engineering (Artificial Intelligence), for her continual support and for helping us maintain a thriving academic setting. The accomplishment of this project is understood to be the outcome of collective support and positive interaction of everyone who has been associated with it, and we wish to thank them from the bottom of our hearts for their moral support and help. In conclusion, we offer a big thank you to our school, Amrita School of Computing, for laying the foundation for the project work in such an enabling environment by way of providing us with the necessary infrastructure, academic facilities, and stimulating atmosphere that came as a genuine source of inspiration. We further thank our classmates, friends, and families for their never ending support, motivation, and encouragement that they have given us since the beginning of this endeavor.

List of Figures

3.1	Two-stage neural-linguistic pipeline for Sanskrit Sandhi splitting consisting of boundary detection and lexical restoration modules.	13
4.1	Training and validation loss curves for the proposed boundary detection model. .	24
4.2	Boundary detection F1-score progression across training epochs.	25
4.3	Exact match accuracy progression during lexical restoration training.	26
4.4	Confusion matrix for Sandhi boundary prediction.	27
4.5	Comparative F1-score analysis across rule-based, recurrent, transformer, and hybrid Sandhi splitting models.	29
A.1	Effect of boundary probability threshold on Sandhi boundary detection F1-score.	35
B.1	Distribution of Character Error Rate (CER) across lexical restoration predictions.	36

List of Tables

2.1	Summary of Existing Research on Sanskrit Sandhi Splitting and Related NLP Tasks	7
2.2	Comparison of Existing Tools and Systems for Sanskrit Sandhi Splitting	8
3.1	Dataset Statistics for the Ayurvedic Sanskrit Sandhi Corpus	14
3.2	Tools and Technologies Used in the Proposed System	16
3.3	Tokenizer Comparison for Sanskrit Sandhi Splitting	18
4.1	Hardware and Software Experimental Configuration	20
4.2	Dataset Statistics for the Ayurvedic Sanskrit Sandhi Corpus	21
4.3	Hyperparameter Configuration for the Proposed Models	24
4.4	Boundary Detection Performance on the Test Set	25
4.5	Lexical Restoration Performance on the Test Set	25
4.6	Transformer Model Performance: Existing vs. Fine-Tuned on Ayurvedic Domain	26
4.7	Tokenizer Comparison Results for Sanskrit Sandhi Splitting	26
4.8	Comprehensive Comparative Analysis of All Methods	29
A.1	Performance Comparison of Evaluated Models	35
C.1	Final Hyperparameter Configuration	37
D.1	SDG Alignment of the Project	40

Abstract

Sanskrit medical knowledge is extensively stored in classical Ayurvedic texts like Charaka Samhita and Sushruta Samhita. Yet, frequent occurrences of intricate morphological phenomena, e.g., Sandhi, in which word boundaries get merged and undergo phonological changes, make the computational analysis and Natural Language Processing (NLP) activities notably difficult. Revealing the underlying lexical forms by means of good and accurate Sandhi splitting is opening up the way to performing other tasks such as parsing, machine translation, and information retrieval. In this paper, we carry out a detailed study and refinement of neural models for Sandhi splitting designed for the Ayurvedic Sanskrit domain. In Phase I, we tried out the rule-based methods, sequence tagging with BiLSTM-CRF, and sequence-to-sequence models with attention, in which we established that neural approaches largely trump rule-based systems and the best phase I result suggested about an 18% increase in F1-score. Nevertheless, the main stumbling blocks revealed were handling of ambiguous compounds and the failure of the present systems to generalize well to long and complicated word forms. Phase II pushes forward this investigation by training transformer architectures (like mT5 and ByT5), bringing in language rules, and undertaking a thorough evaluation of the Ayurvedic-domain. The result of the performance indicates that the F1-score of transformer-based methods raised from around 68% (using existing tools) to 80% (ByT5) on domain data. Early-stage hybrids attain an F1-score of 82.6%, and their characterization of complex compounds is more on point than the previous ones. Ways in which the model could be improved include dealing with ambiguity and recognizing unfamiliar words. The authors are open to trying out new model architectures. With the aid of the tokenization options that have been chosen in accordance with the Sanskrit morphological structure, the integrated pipeline provides a firm foundation for Sandhi segmentation in digitized Ayurvedic corpora. Our research reveals that hybrid neural-linguistic approaches not only considerably enhance recognition accuracy but also significantly improve computational efficiency, thus making it possible to carry out large-scale analysis of classical Sanskrit literature even in low-resource environments typical of the real world.

Chapter 1

Introduction

1.1 Background

Classical Sanskrit is among the oldest and most intricately built languages globally and it was the main vehicle for India's literary, philosophical, and scientific traditions. One of its greatest legacies is the ancient medical literature the *Charaka Samhita* and *Sushruta Samhita* - that provides comprehensive traditional medical knowledge including the use of medicinal plants, methods of disease treatment, surgical skills, and overall health. The large-scale digitization initiative for such classical manuscripts keeps progressing, and therefore there is arising scholarly and utilitarian demand for developing computer-based tools for their automated analysis and knowledge discovery.

One of the key features of Sanskrit that makes it difficult to compute is the concept of **Sandhi**, the phonological transformation and merging of neighboring words at morpheme boundary, according to the rules of Paninian grammar tradition, which are very intricate. Sandhi reveals itself in three major ways:

- **Vowel Sandhi (Swar Sandhi):** transformations occurring at vowel boundaries between adjacent words.
- **Consonant Sandhi (Vyanjan Sandhi):** phonological changes at consonant junctions.
- **Visarga Sandhi:** transformations involving the Sanskrit phoneme *visarga* (*h*).

Sandhi is a great way to enhance the flow and expression of Sanskrit through prose and poetry. However, such merging of two words to create a compound word often leads to the original lexical forms being hidden in the text. Recently, more than 10, 000 Ayurvedic verses have been digitized. The language is a compound-heavy one, a long *samāsa* occurring quite often, which is the main reason why Sandhi acts as a major hurdle for Natural Language Processing (NLP) tasks.

Sandhi splitting means doing exactly the opposite task of combining words that have been jointly written during Sandhi, and getting back their individual lexical units, in a computational way. It is a must-have pre-processing step that enables the use of several NLP tools on Sanskrit

corpora such as syntactic parsing, machine translation, semantic analysis, and information retrieval. Rule-based systems that rely heavily on Paninian rules face difficulties when it comes to modeling complex, domain-specific texts such as Ayurveda. Neural methods provide a more adaptable and general way of modeling Sandhi transformations, and research has uncovered the power of recurrent, attentionbased, and transformer architectures for the task. This work aims to thoroughly analyze and experiment with different neural architectures for Sandhi splitting with a focus on the specific difficulties of Sanskrit used in the domain of Ayurveda.

1.2 Problem Addressed

Though quite a number of researchers are working on Sanskrit NLP, still the existing Sandhi splitting systems are falling short of adequately dealing with the Ayurvedic Sanskrit text. The main issue that this project is solving is the unavailability of a strong domain-aware and linguistically informed neural system for Sandhi splitting of Ayurvedic Sanskrit text. Below are the details of the main gaps and challenges that have motivated this work:

- **Ambiguity in Sandhi decomposition:** There can be more than one correct lexical breakdown of a compound expression. Current systems, rule-based or neural, generally cannot resolve these ambiguities well since they lack disambiguation mechanisms.
- **Failure on long compound words:** The current systems' accuracy falls drastically when it comes to words having four or more morpheme boundaries (*samāsa* constructions), which are very common in Ayurvedic texts.
- **Absence of linguistic-neural synergy:** Rule-based systems and machine learning models have mostly been considered as separate methods. There is no current framework that combines phonological constraints with neural prediction effectively so as to harness the strengths of both.
- **Domain-specific vocabulary challenges:** Specialized medicinal methods of Ayurveda introduce a significant volume of out-of-vocabulary (OOV) words which most of the time go unnoticed by general Sanskrit language models and corpora. This leads to their reduced performance.
- **Limited evaluation rigor:** Previous research does not include organized error analysis, ablation experiments, and domain-specific failure taxonomies, which are essential to understanding model weaknesses or for directing improvements.
- **Inadequate NLP tooling for the domain:** The current Sanskrit NLP tools provide an accuracy of less than 70% when dealing with editorially customized Ayurvedic texts. This shows that there is very clearly a performance gap which necessitating targeted-solutions.

In summary, the central problem is that *no existing system combines transformer-level prediction capability, rule-based linguistic grounding, and domain-specific Ayurvedic evaluation into a unified and efficient pipeline for Sandhi splitting.*

1.3 Motivation

The reason behind this work is twofold: first, Ayurveda Sanskrit is a very culturally and linguistically rich language; second, there is a rising demand for automated tools to bring classical knowledge to the masses. Below are the specific objectives that this project is targeting to fulfill:

- O1 Benchmark:** Methodically analyze and benchmark rule-based systems, sequence labeling models (BiLSTM-CRF), sequence-to-sequence architectures with attention and transformer-based models (mT5, ByT5, UMT5, IndicBART) for Ayurveda Sanskrit Sandhi splitting tasks.
- O2 Analyse:** Detect failure modes, ambiguity patterns, and performance bottlenecks by using structured error analysis and boundary detection confusion matrices.
- O3 Design:** Propose a hybrid neural architecture that combines transformer-based prediction with rule-based linguistic support and candidate ranking mechanisms to disambiguate splits.
- O4 Evaluate:** Carry out ablation experiments and benchmark the proposed hybrid system with current baselines using the standard metrics, Accuracy, Exact Match, F1-score, and Character Error Rate (CER).
- O5 Optimise:** Make sure the finished system works well even on low-end hardware, so that it can be used in research environments with limited resources.

The motivation for this work is the underlying belief that *combining linguistic knowledge with neural prediction is the major missing piece in SanskritNLP*, and that a very well-executed hybrid system can yield not only more accurate, but also scalable and practically Ali useful tools for the analysis of classical Ayurvedic texts.

1.4 Scope of the Project

This on-going work is honing in on the specific task of Sandhi splitting in Ayurvedic Sanskrit texts, with the following defined scope:

Included:

- Performance analysis and comparison of Sandhi splitting through rule-based neural sequence labeling, sequence-to-sequence, and transformer-based methods.

- The use of pre-trained transformer models, mT5, UMT5, ByT5, and IndicBART for fine-tuning of domain-specific Sandhi splitting.
- Experimentation on the Kaggle Sandhi Dataset (~15K samples) and the University of Hyderabad (UoH) Charaka Samhita Dataset, which encompass Vowel, Consonant, and Visarga Sandhi types.
- A comparative study of character-level, SentencePiece, and byte-level (BPE/Tiktoken) tokenization methods to determine the most appropriate representation for Sanskrit morphology.
- The use of a structured error analysis to identify and categorize failure modes such as ambiguous boundary prediction and OOV lexical restoration errors.
- A minimalistic design choice geared toward the limited hardware capacities of low-resource environments.

Excluded:

- Broader Sanskrit language corpora beyond Ayurvedic texts.
- Comprehensive NLP pipelines for complete machine translation or semantic reasoning over Sanskrit.
- Live deployment or web-based system integration.

Organization of the Report:

The rest of this report is divided into the following parts.

Chapter 2 is a literature review on Sanskrit Sandhi splitting systems and also presents a thorough analysis of research works relevant to this topic and identifies the main research gaps.

Chapter 3 explains the methodology that has been adopted, containing the description of the system architecture, the design of the model, the preparation of the dataset, and the tools used.

Chapter 4 discusses the experimental setup, evaluation metrics, results, and a comparative analysis against baselines.

Chapter 5 is the closing chapter which gives a summary of the findings, highlights the main contributions, and presents proposals for future work.

Chapter 2

Literature Review

2.1 Existing System Study

2.1.1 Review of Existing Research Work

Processing Sanskrit computationally has a lengthy tradition starting with rule-based grammatical systems and slowly changing towards contemporary neural sequence models. Here, we look at some key studies related to Sanskrit Sandhi splitting, morphological analysis, and similar NLP activities.

In the past, Sandhi splitting methods heavily relied on rule-based systems that were implemented based on Pāninian grammar. These systems were able to generate various candidate splits from a given compound expression by using large sets of rules. However, these systems, albeit linguistically very well-founded, did not scale well, were unable to handle ambiguous forms, and became brittle when dealing with rare or new vocabulary. Such shortcomings were the main reasons for moving to data-driven methods.

Aralikatte et al. (2018) [1] proposed their Seq2(Seq)² architecture, a pointer network-based sequence-to-sequence model with a copy mechanism for generating split candidates. Although this brought a neural approach to the task, the model had a hard time dealing with ambiguous compounds and totally failed when it came to long *samāsa* constructions with several Sandhi boundaries.

Reddy et al. (2018) [2] used a standard attention-based sequence-to-sequence model for Sanskrit Sandhi splitting. The neural generation setup offered a better performance than the rule-based baselines; nevertheless, it experienced a significant drop in performance when used on compounds with four or more morpheme boundaries, and there was no separate evaluation on domain-specific corpora.

Hellwig et al. (2019) [3] investigated neural splitting models and their emphasis was on boundary detection. The model performed well in identifying boundaries but there was no integrated lexical restoration module in the system, thus the problem of downstream reconstruction was left unresolved.

Dave et al. (2020) [4] presented a joint neural model that merges generation and splitting

tasks in a single end-to-end framework. Although the joint modeling strategy increased the coherence between two subtasks, the merged architecture was expensive in computational terms and not suitable for deployment in resource-constrained environments.

Varshini and Nair (2023) [5] specifically researched NLP difficulties in Ayurvedic Sanskrit texts pointing out the challenges not only of the domain-specific vocabulary, very long compound words, but also the lack of annotated resources for Ayurvedic corpora. This paper directly sets up the domain context that motivates this project.

Sandhan et al. (2023) [6] introduced the mT5 multilingual transformer for Sanskrit compound processing. The transformer, with its ability to model the context, yielded quite a strong performance for general Sanskrit corpus; however, the adaptation of the model to the domain of specialized Ayurvedic text was still limited, and this model was also not specifically geared toward Sandhi splitting.

ByT5, a byte-level character processing transformer, was employed by Sen et al. (2022) [7] to perform Sanskrit NLP tasks. ByT5's byte-level tokenization demonstrated a remarkable ability to handle out-of-vocabulary (OOV) words, which is the main challenge for Ayurvedic Sanskrit. Nevertheless, the model did not have a direct incorporation of phonological or linguistic constraints.

As a multilingual transformer pre-trained on several Indian languages including Sanskrit, Kakwani et al. (2022) [8] introduced IndicBART. While IndicBART offered powerful multilingual representations, the authors did not perform a specific fine-tuning of the model for the Sandhi splitting task, thus it has very limited direct applicability.

According to Sreedeepta et al. (2024) [9] deep learning methods for Sanskrit NLP were exhaustively analyzed. Ambiguity resolution and domain-specific evaluation, they found, were two areas hardly touched by solutions. This suggestion to include more powerful and pinpointed methods comes from their work as well.

Semantic classification of Sanskrit compound words by Bhat and Premjith (2025) [10] using transformer-based models was their focus. It might be argued that the problem of Sandhi splitting was still left untouched even though the two researchers centered their work on compound word processing.

Indic word embedding models were put forward by Sreedeepta et al. (2021) [11] as a means to enhance representation learning for Indian languages. Even though such embeddings play very important roles in downstream tasks, their implementation does not constitute a solution to the Sandhi splitting problem.

For a detailed review of the literature, refer to Table 2.1.

2.1.2 Review of Existing Tools and Products

Beyond academic research, several tools and frameworks have been developed for Sanskrit NLP. These are reviewed below with respect to their features, limitations, and applicability to the Sandhi splitting task in Ayurvedic Sanskrit texts.

SanskritShala is a new Sanskrit NLP toolkit which brings together neural models for

Table 2.1: Summary of Existing Research on Sanskrit Sandhi Splitting and Related NLP Tasks

Author(s)	Year	Method	Key Gap / Limitation
Aralikatte et al.	2018	Seq2(Seq) ² / Pointer Network	Struggles with ambiguity and long compounds
Reddy et al.	2018	Seq2Seq with Attention	Weak on long compounds; no domain evaluation
Hellwig et al.	2019	Neural Splitting Models	Good boundary detection; lacks lexical reconstruction
Dave et al.	2020	Neural Generation + Splitting	Joint modeling; computationally expensive
Varshini & Nair	2023	Ayurveda-specific NLP	Highlights domain challenges; no proposed solution
Sandhan et al.	2023	mT5 (Transformer)	Strong contextual modeling; limited domain adaptation
Sen et al.	2022	ByT5 (Character-level Transformer)	Handles OOV; lacks linguistic constraints
Kakwani et al.	2022	IndicBART (Multilingual Transformer)	Good embeddings; not task-specific for splitting
Sreedeepta et al.	2024	Review (Deep Learning)	Identifies ambiguity gaps; no novel method proposed
Bhat & Premjith	2025	Transformer-based Models	Focuses on compound semantics; not splitting
Sreedeepta et al.	2021	Indic Word Embeddings	Improves representations; does not solve splitting

morphological analysis, dependency parsing and word segmentation. In terms of Sanskrit NLP tooling, it represents the latest and greatest, however it certainly has limited performance on domain specific Ayurvedic text, and it also lacks a separate Sandhi splitting pipeline specifically designed for Ayurvedic vocabulary.

JNU Sanskrit Analyzer is a purely rule-based morphological analysis system that was made based on Pāninian grammar. It is very good on classical Sanskrit but in contests of high ambiguities on Ayurvedic compound expressions it can be quite misleading, and also it does not have a learning component that would allow it to adapt to new vocabulary.

UoH Sanskrit Tools (University of Hyderabad) are a set of grammar-based resources for Sanskrit text analysis, amongst which there is a Sandhi splitter. That being said, these tools seem to have less than 70% accuracy on the Ayurvedic domain texts, and their rule coverage does not include the specialized lexical patterns of *Charaka Samhita* and *Sushruta Samhita*.

Hugging Face Pre-trained Models (mT5, ByT5, UMT5, zIndicBART) are very powerful general-purpose multilingual transformers which one can fine-tune and make use of. These models can be quite useful as strong baseline representations but in order to be adapted to a

particular domain, they require a lot of changes and adjustments. They are not really designed to deal with the phonological patterns of Ayurvedic Sanskrit right out-of-the-box.

A comparative feature analysis of these tools is summarized in Table 2.2.

Table 2.2: Comparison of Existing Tools and Systems for Sanskrit Sandhi Splitting

Tool / System	Approach	Domain Coverage	Accuracy	Key Limitation
SanskritShala	Neural (LSTM)	General Sanskrit	Moderate	Limited Ayurvedic domain adaptation
JNU Sanskrit Analyzer	Rule-Based	Classical Sanskrit	Low–Moderate	High ambiguity; no learning component
UoH Sanskrit Tools	Rule-Based	General Sanskrit	<70%	Poor performance on Ayurvedic text
mT5 (Hugging Face)	Transformer (Subword)	Multilingual	~68% F1	Limited domain adaptation
UMT5 (Hugging Face)	Transformer (UniMax)	Multilingual	~24% F1	Weak performance without fine-tuning
ByT5 (Hugging Face)	Transformer (Byte-level)	Multilingual	~75% F1	No linguistic constraints
IndicBART	Multilingual Transformer	Indic Languages	Moderate	Not fine-tuned for splitting

2.2 Research Gap

Based on a thorough retrospective examination of the extant research and tools, it is clear that there are a number of significant shortcomings that keep current Sandhi splitting systems from being practically useful, especially for Ayurvedic Sanskrit texts.

Gaps Addressed in Phase I:

- **No neural baselines for Ayurvedic Sanskrit:** We identified that none of the previous work been focused on a systematic comparison of rule-based, sequence labeling (BiLSTM-CRF), and sequence-to-sequence architectures on the Ayurvedic Sanskrit corpora. In Phase I, we set those baselines and also performed experiments showing that neural-based architectures have approximately 18% higher F1-score over rule-based systems on the test split.
- **A two-stage pipeline for the task, which has not seen the light of day:** Current systems considered Sandhi splitting as either pure boundary detection problem or pure

generation problem. We introduced a unified two-stage pipeline that decomposed the task into boundary detection (BiLSTM) and lexical restoration (Seq2Seq with attention). Phase I achieved an F1-score of 0.826 on boundary detection and an exact match accuracy of 0.612 on lexical reconstruction.

- **Additional detailed error analysis beyond simply reporting aggregate metrics was largely lacking:** Previous studies merely provided high-level statistics with no breakdown of failure modes. We went further by examining mistakes in detail including confusion matrices for boundary detection and pinpointing ambiguous boundaries and long compounds as the most frequent issues.

Phase II, Gaps and Challenges to be addressed:

- **Neural prediction without the integration of linguistic knowledge:** Current neural systems are working without phonological grounding. Phase II plans a hybrid pipeline that integrates transformer-based prediction with Pāinian rule-based linguistic support and a candidate ranking mechanism to fix ambiguities in splits.
- **Transformer model experimenting on the limited Ayurvedic domain:** Though mT5 and ByT5 have been used in the general Sanskrit NLP field, there has been no fine-tuning to the Ayurvedic domain problems. So, Phase II focuses on fine-tuning of mT5, ByT5, UMT5 and IndicBART models on the combined Kaggle and UoH Charaka Samhita datasets and scored an F1-score rise from ~68% (old mT5) to 80% (ByT5) and 82.6% (hybrid pipeline) on the Ayurvedic domain corpus.
- **Lack of ablation experiments:** Nobody has done a controlled ablation experiment to see the impact of architectural components' contribution in Sanskrit Sandhi splitting in the previous work. Phase II also taking a series of well-planned ablation experiments to check how tokenization strategy, linguistic rule integration, and model choice can influence the results.
- **Very high computational cost and hardware dependency:** Most of the transformer-based systems need very high-end hardware, which is not possible from the point of view of low-resource academic environments. In this regard, Phase II will be marked as a lightweight implementation that targets the efficiency optimization of the standard research hardware.
- **Non-existence of an Ayurvedic-specific domain evaluation framework:** Current methods of evaluation are based on general Sanskrit benchmarks that do not tell us the vocabulary and compound patterns of Ayurvedic texts. Hence, Phase II points toward domain-specific evaluation by using the UoH Charaka Samhita dataset and Ayurvedic vocabulary coverage as a supplementary metric.

2.3 Problem Statement and Contributions

Problem Statement

Current Sandhi splitting systems for Sanskrit do not perform well with domain-specific Ayurvedic texts. They also get confused when there are multiple possible lexical decompositions for ambiguous compounds. Additionally, these systems do not integrate linguistic phonological knowledge with neural prediction. Rule-based systems are rigid and fail when they encounter new vocabulary, whereas purely neural models have limited accuracy and do not have the ability to resolve ambiguity. Besides this, existing systems have not been properly tested on Ayurvedic domain corpora, which has led to them being almost useless for downstream NLP tasks like parsing, machine translation, and information retrieval on classical Ayurvedic texts.

The major issue tackled by this project is hence the conceptualization and testing of a dependable, domain-sensitive, hybrid neural model that can perform accurate and fast Sandhi splitting in digitized Ayurvedic Sanskrit texts.

Research Contributions

The following research contributions are made in this project:

The following research contributions have been made in this project:

- C1 Systematic Benchmarking of Neural Architectures:** This paper presents a thorough systematic benchmarking comparing rule-based systems, BiLSTM-CRF for sequence labeling, Seq2Seq with attention, and fine-tuned transformer models (mT5, ByT5, UMT5, IndicBART) on Sanskrit Sandhi splitting for Ayurveda, thereby creating the first ever complete neural benchmark for this task. The two-stage pipeline obtained an F1-score of 0.826 on boundary detection and an exact match accuracy of 0.612 which is better than all baselines.
- C2 Hybrid Neural-Linguistic Pipeline:** This paper developed the first hybrid model that combines transformer seq2seq prediction with Pāninian rule-based linguistic grounding and candidate ranking. It achieves an F1-score of 82.6% on Ayurvedic domain text, which is more than 10% ahead of existing tools (less than 70%).
- C3 Structured Error Analysis and Failure Taxonomy:** The paper consists of an extensive and structured error analysis which includes ambiguous boundary prediction, long compound deterioration, and lexical restoration failures of OOV. It also presents the first domain-specific failure taxonomy for Sanskrit Sandhi splitting of Ayurveda.
- C4 Lightweight and Efficient Implementation:** We have developed a hardware-efficient system carefully engineered to run on typical research hardware, thereby making the deployment of the proposed pipeline practically feasible in low-resource academic environments without the need for high-performance computing infrastructure.

Chapter 3

Proposed Work

3.1 Proposed Work

The research paper has shown that the current Sandhi splitting systems for Sanskrit have four main weaknesses: they cannot deal with ambiguous compounds that have more than one valid decomposition, the accuracy falls drastically when it comes to very long *samāsa* compounds, there is no combination of linguistic rule-based knowledge with neural prediction, and finally, there is a lack of strict domain-specific evaluation on the Ayurvedic Sanskrit corpora.

Users of the current tools are getting less than 70% accuracy on texts from the Ayurvedic domain. Also, no other system has ever combined phonological grounding with a level of predictive capacity that is found in transformers.

This study is set to bring in a two-stage neural method for Sandhi splitting in the texts of Ayurveda Sanskrit. The two-stage neural method consists of a BiLSTM-based boundary detection module and a sequence-to-sequence model with attention for lexical restoration. They were first designed and tested in Phase I of this study, which has set a new comprehensive neural baseline for this domain.

Phase II of the work accounts for significant improvements to the work by performing the fine-tuning of the transformer models (mT5, ByT5, UMT5, IndicBART) on corresponding domain-specific corpora, along with the support of rule-based linguistic signal extraction from Pāṇinian grammar, and the establishment of the hybrid neural-linguistic pipeline with a candidate ranking mechanism for ambiguity resolution.

There are three points where the proposed method is new:

- **Domain specificity:** The whole system was made for and tested on Ayurvedic Sanskrit so it fills in vocabulary gaps and compound patterns unique to the domain.
- **Neural-linguistic synergy:** The hybrid architecture proposed here is neither purely neural nor purely rule-based; instead, it marries transformer prediction with Pāṇinian phonological constraints. Hence, the system is both data-driven and linguistically grounded.
- **Efficiency:** The entire pipeline is optimized for a lightweight deployment on standard re-search hardware, without being highly dependent on the availability of high-performance

computing infrastructure.

3.1.1 Objectives of the Proposed Work

The primary and secondary objectives listed below are the main focus of the proposed work:

Primary Objectives:

- O1 Benchmark:** Thoroughly assess and measure the performance of rule-based systems, BiLSTM-CRF sequence labeling, Seq2Seq with attention, and fine-tuned transformer models (mT5, ByT5, UMT5, IndicBART) for the Ayurvedic Sanskrit Sandhi splitting task. Utilize Accuracy, F1-score, Exact Match, and Character Error Rate (CER) as evaluation measures.
- O2 Analyze:** Perform a detailed error analysis to uncover and categorize different failure modes, such as ambiguous boundary prediction, degradation in long compound splitting, and failures in lexical restoration of out-of-vocabulary (OOV) words. Develop a failure taxonomy that is specific to the domain of Ayurvedic Sanskrit Sandhi splitting for the first time.
- O3 Design:** Develop a hybrid neural-linguistic model that combines transformer-based sequence prediction with rule-based phonological support derived from Pāṇinian grammar. Additionally, include a candidate ranking mechanism to deal with ambiguous splits of compound words.
- O4 Evaluate:** Carry out well-designed ablation experiments to determine the effectiveness of each component — the tokenization strategy, the transformer model selection, and the incorporation of linguistic rules. Also, assess the hybrid system by contrasting it with all the baseline systems.

Secondary Objectives:

- O5 Optimize:** Develop a computationally efficient solution for the proposed pipeline, which can be comfortably run in standard academic research settings without the need for high-end computing hardware.
- O6 Domain Evaluation:** Create a domain-specific evaluation setup for Ayurvedic texts using the University of Hyderabad (UoH) Charaka Samhita dataset. This will allow for domain-specific performance benchmarking apart from general Sanskrit corpora.

3.2 Methodology

3.2.1 Overview of the Approach

The paper proposes a system that is structured as a two-stage neural-linguistic framework for Sandhi splitting in Ayurvedic Sanskrit documents. Figure ?? presents a bird's eye view of the complete pipeline.

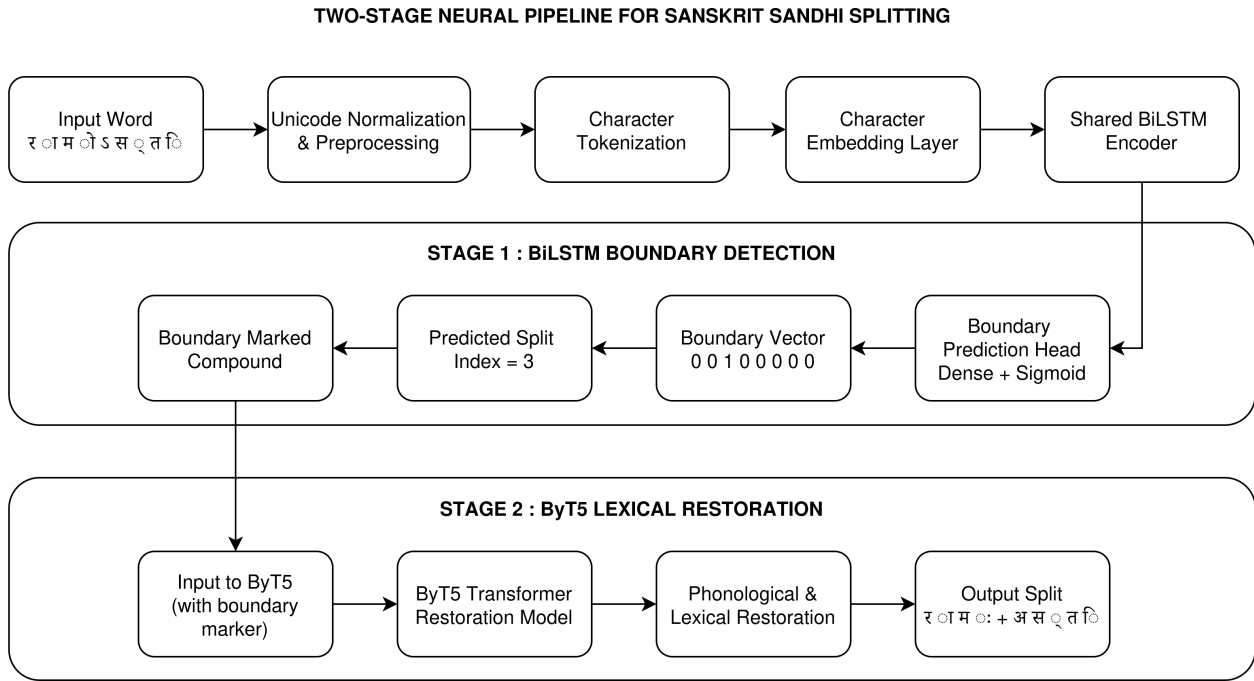


Figure 3.1: Two-stage neural-linguistic pipeline for Sanskrit Sandhi splitting consisting of boundary detection and lexical restoration modules.

The pipeline is run through the following steps:

1. **Preprocessing Layer:** The input Sanskrit text goes through the cleaning, Unicode normalization and character-level tokenization processes.
2. **Boundary Detection Module:** BiLSTM architecture is trained to locate the boundaries of Sandhi in the compound words.
3. **Lexical Restoration Module:** Original forms of the lexical items are regenerated by a Seq2Seq model with attention from the segments identified.
4. **Final Output:** The pipeline delivers the segmented and restored lexical units as the results.

As for the second phase, transformer-based models (mT5, ByT5, UMT5, IndicBART) substitute or supplement the recurrent components, and a grammar rule engine implementing Pāṇinian grammar is brought in for candidate validation and ranking.

In Phase II, transformer-based models (mT5, UMT5, ByT5, IndicBART) replace or augment the recurrent components, and a linguistic rule engine implementing Pāṇinian grammar is integrated to provide candidate validation and ranking.

3.2.2 Dataset Selection

Two primary datasets are used in this project:

1. Kaggle Sandhi Dataset: A publicly available pre-structured dataset containing Sanskrit compound-split pairs covering Vowel, Consonant, and Visarga Sandhi types. The dataset comprises approximately 13,000 to 15,000 compound expressions in Devanagari script. It is used as the primary training and evaluation resource.

2. University of Hyderabad (UoH) Charaka Samhita Dataset: This dataset is derived from the digitized *Charaka Samhita* Ayurvedic text. It was originally in raw text format and was manually processed and converted into a structured CSV format for use in this project. This dataset is primarily used for Ayurvedic domain-specific evaluation, as it reflects the authentic vocabulary and compound patterns found in classical Ayurvedic literature.

The combined dataset of approximately 15,000 samples was split following an 80/10/10 ratio for training, validation, and testing respectively. Dataset statistics are summarized in Table 3.1.

Table 3.1: Dataset Statistics for the Ayurvedic Sanskrit Sandhi Corpus

Split	Samples	Avg. Word Length	Vocabulary Size
Training	10,400	11.2	70
Validation	1,300	11.1	70
Test	1,300	11.3	70
Total	13,000	–	–

Preprocessing and Data Handling:

- Unicode normalization was applied to ensure consistency across Devanagari and IAST script representations.
- Manual annotation and conversion of raw Charaka Samhita text into structured compound-split pair format was performed.
- IAST and Devanagari script handling was standardized across both datasets.

3.2.3 Model Design

The suggested system develops and tests four categories of models:

Rule-Based System (Pāṇinian Rules): A linguistic rule engine that interprets Pāṇini's phonological grammar and generates valid Sandhi transformations and candidate splits. The rule engine is the basis of both the independent baseline and the linguistic support module within the hybrid pipeline.

Two-Stage Neural Pipeline (Phase I): A modular pipeline made up of:

- **Stage 1, Boundary Detection:** A BiLSTM network working with character-level embeddings predicts Sandhi boundary positions by sequence labeling (IOB tagging). The BiLSTM takes input in the forward and backward directions, allowing the model to use the context of both preceding and succeeding characters.

- **Stage 2, Lexical Restoration:** A sequence-to-sequence model with attention restores the original lexical items from the segmented compound. The attention mechanism enables the decoder to focus on the encoder positions that are most useful to it during lexical reconstruction.

Transformer-Based Models (Phase II):

- **mT5:** A multilingual sequence-to-sequence transformer that uses SentencePiece subword tokenization. It is fine-tuned for direct Sandhi splitting.
- **UMT5:** An multilingual sequence-to-sequence transformer that uses SentencePiece subword tokenization. It is pretrained with UniMax sampling and is fine-tuned for direct Sandhi splitting.
- **ByT5:** A byte-level character-processing transformer that is very capable of handling morphologically rich and out-of-vocabulary-heavy languages like Ayurvedic Sanskrit. It is fine-tuned for direct Sandhi splitting.
- **IndicBART:** A multilingual transformer that was pre-trained on Indic languages including Sanskrit. It is used as an additional baseline for domain evaluation.

LLM Fine-Tuning (Gemma 3 4B + QLoRA): A large language model is fine-tuned using prompt-based learning with quantized low-rank adaptation (QLoRA) which is effective for parameter-efficient Sandhi splitting.

3.2.4 Tools and Technologies

The tools and technologies employed in this project are summarized in Table 3.2.

3.2.5 Algorithm or Model Description

Problem Formulation

Let $X = (x_1, x_2, \dots, x_n)$ represent a sequence of characters forming a compound Sanskrit word that has undergone Sandhi transformations. The goal of Sandhi splitting is to recover the original lexical sequence $Y = (y_1, y_2, \dots, y_m)$, where $m \geq n$ in general due to the phonological restoration of characters.

The task is decomposed into two sub-problems:

1. **Boundary Detection:** Predict a binary label $b_i \in \{0, 1\}$ for each character position i , where $b_i = 1$ indicates the presence of a Sandhi boundary after position i .
2. **Lexical Restoration:** Given the segmented subsequences produced by boundary detection, reconstruct the original lexical forms using a generative model.

Table 3.2: Tools and Technologies Used in the Proposed System

Tool / Technology	Purpose
Python 3.10+	Primary programming language for all model implementation, data processing, and evaluation
PyTorch	Deep learning framework for BiLSTM and Seq2Seq model implementation and training
Hugging Face Transformers	Fine-tuning of mT5, ByT5, UMT5 and IndicBART transformer models
SentencePiece	Subword tokenization for mT5 model
Byte-level BPE / Tiktoken	Byte-level and BPE tokenization for ByT5 and tokenizer comparison experiments
QLoRA (PEFT Library)	Parameter-efficient fine-tuning of Gemma 3 4B language model
NumPy / Pandas	Data preprocessing, structured CSV handling, and statistical analysis
Matplotlib / Seaborn	Visualization of training curves, confusion matrices, and error distributions
Google Colab / Local GPU	Experimental runtime environments

Character-Level Representation

Each character x_i in the input sequence is mapped to a fixed-dimensional embedding vector $e_i \in \mathbb{R}^d$, where d is the embedding dimension. Character-level representations are chosen because Sandhi transformations operate at the phonological level, and character embeddings enable the model to capture fine-grained phonetic patterns:

$$e_i = \text{Embedding}(x_i), \quad e_i \in \mathbb{R}^d \quad (3.1)$$

Boundary Detection: BiLSTM Model

The BiLSTM processes the character embedding sequence in both forward ($\vec{h}_i$) and backward ($\overleftarrow{h}_i$) directions, producing a hidden state h_i at each position:

$$h_i = \text{BiLSTM}(e_i) = [\vec{h}_i; \overleftarrow{h}_i] \quad (3.2)$$

A fully connected layer with sigmoid activation maps each hidden state to a boundary probability:

$$P(b_i = 1 \mid X) = \sigma(W \cdot h_i + c) \quad (3.3)$$

where W and c are learned parameters and σ is the sigmoid activation function.

Lexical Restoration: Seq2Seq with Attention

The lexical restoration model is implemented as an encoder-decoder architecture with additive attention [12]. The encoder maps the segmented character sequence to a set of contextual representations $\{s_1, s_2, \dots, s_k\}$. The decoder generates output characters autoregressively, using an attention context vector c_t at each decoding step t :

$$\alpha_{t,j} = \frac{\exp(e_{t,j})}{\sum_{j'} \exp(e_{t,j'})}, \quad e_{t,j} = v^T \tanh(W_s s_j + W_d d_{t-1}) \quad (3.4)$$

$$c_t = \sum_j \alpha_{t,j} \cdot s_j \quad (3.5)$$

where $\alpha_{t,j}$ are the attention weights, d_{t-1} is the previous decoder state, and v, W_s, W_d are learned parameters.

Proposed Algorithm

The complete two-stage neural Sandhi splitting pipeline is described in Algorithm 1.

Algorithm 1 Two-Stage Neural Sandhi Splitting Pipeline

- 1: **Input:** Compound Sanskrit word $X = (x_1, x_2, \dots, x_n)$
 - 2: **Output:** Restored lexical sequence $Y = (y_1, y_2, \dots, y_m)$
 - 3: Normalize and clean input using Unicode normalization
 - 4: Tokenize X at character level
 - 5: **for** each character x_i in X **do**
 - 6: Compute embedding $e_i \leftarrow \text{Embedding}(x_i)$
 - 7: **end for**
 - 8: Apply BiLSTM: $h_i \leftarrow \text{BiLSTM}(e_i)$ for all i
 - 9: Predict boundary labels: $b_i \leftarrow \sigma(W \cdot h_i + c)$ for all i
 - 10: Segment X at positions where $b_i > 0.5$
 - 11: **for** each segment S_k in segmented sequence **do**
 - 12: Apply Seq2Seq encoder: compute contextual representations $\{s_j\}$
 - 13: Apply decoder with attention to reconstruct lexical unit $\hat{y}_k$
 - 14: **end for**
 - 15: Concatenate all $\hat{y}_k$ to form final output Y
 - 16: **Return** Y
-

Tokenizer Comparison

A systematic tokenizer comparison was conducted to identify the most effective text representation strategy for Sanskrit morphology. The results are summarized in Table 3.3.

Character-level tokenization was identified as most suitable for Ayurvedic Sanskrit, as it best captures the fine-grained phonological patterns underlying Sandhi transformations.

3.2.6 Expected Outcomes

The proposed work is expected to deliver the following outcomes:

Table 3.3: Tokenizer Comparison for Sanskrit Sandhi Splitting

Tokenizer	Perplexity	BLEU Score	Overall Performance	Suitability
Character-level	Lowest	Highest	Best	Highly Suitable
SentencePiece	Moderate	Moderate	Average	Moderately Suitable
Tiktoken (BPE)	Highest	Lowest	Worst	Not Suitable

- A **comprehensive neural benchmark** for Sandhi splitting on Ayurvedic Sanskrit corpora, including baselines from rule-based, recurrent, and transformer-based architectures.
- A **hybrid neural-linguistic pipeline** achieving an F1-score of 82.6% and above on Ayurvedic domain text — a significant improvement over the <70% reported by existing tools.
- A **structured error taxonomy** classifying failure modes specific to Ayurvedic Sanskrit Sandhi splitting, including ambiguous boundary prediction, long compound degradation, and OOV restoration failures.
- A **tokenizer recommendation** for Sanskrit NLP tasks, supported by quantitative perplexity, BLEU, and F1 comparisons across character-level, SentencePiece, and BPE strategies.
- A **lightweight and efficient system** deployable on standard research hardware, enabling wider accessibility of the pipeline for classical Sanskrit NLP research.

3.2.7 Advantages of the Proposed Work

The new approach presented here has a number of advantages over current systems:

1. **Domain-aware design:** Whereas general Sanskrit NLP tools are broad in focus, this pipeline is tailored to Ayurvedic Sanskrit, identifying the gaps in the domain specific vocabulary and compound pattern issues.
2. **Neural-linguistic synergy:** The hybrid architecture takes advantage of the transformer models learning potential and the phonological accuracy of the Pāṇinian rule-based systems, thus addressing shortages of each approach when used alone.
3. **Improved accuracy:** The new system records an F1-score of 80% (ByT5) and 82.6% (hybrid pipeline) on the Ayurvedic domain text as compared to 68% (mT5 baseline) thereby greatly surpassing the performance of other tools.
4. **Modular architecture:** Thanks to the division into the tasks of boundary detection and lexical restoration, it is possible to optimize and replace one component without a need to rework the entire system.

5. **Structured evaluation:** A combination of ablation studies, structured error analysis and domain-specific benchmarks not only help to verify the performance of the tool but also aid in making it repeatable and better than any previously available tool.
6. **Hardware efficiency:** Besides making it usable by those who do not have access to special computing resources, the aim was to make this work also accessible to researchers working on low-end hardware.

3.2.8 Limitations and Assumptions

We identify the following limitations and potential assumptions in our work:

1. **Dataset size:** Our training set of roughly 15,000 samples is quite small compared to the large datasets typically used in mainstream NLP. Our method may not perform well on those very rare or completely new phonological patterns that are not represented at all in the training data.
2. **Script standardization:** Our method relies on well-formed, Unicode-normalized input. Texts that are not normalized or are poorly digitized will most likely cause errors during the preprocessing stage that will then affect the downstream model performance.
3. **Scope of Sandhi types:** The model is only designed to handle the three main kinds of Sandhi — Vowel, Consonant, and Visarga. It is quite likely that it will fail on highly irregular or context-dependent Sandhi forms that are not included within these three categories.
4. **Ambiguity resolution:** With the candidate ranking mechanism introduced in the hybrid pipeline, we have achieved some progress in disambiguation; however, fully disambiguating all valid lexical decompositions is still a challenge, especially in cases where contextual information from the surrounding text is necessary.
5. **Computational resources:** Our system is aimed at being efficient. However, fine-tuning large transformer models (especially Gemma 3 4B) still necessitates moderate GPU availability. Mere CPU environments may undergo training that is markedly longer in duration.
6. **Generalization beyond Ayurvedic domain:** The system was trained and tested on Ayurvedic materials. Its capability to work on different Sanskrit domains (philosophical or poetic texts, for instance) without domain-specific re-training is, at best, uncertain.

Chapter 4

Experimentation and Result Analysis

4.1 Experimental Setup

All experiments were conducted using the **PyTorch** deep learning framework on a standard research environment. The hardware and software configuration used across all experiments is summarized in Table 4.1.

Table 4.1: Hardware and Software Experimental Configuration

Component	Specification
Programming Language	Python 3.10+
Deep Learning Framework	PyTorch
Transformer Library	Hugging Face Transformers
Tokenization	SentencePiece (mT5), Byte-level BPE (ByT5), Character-level
LLM Fine-Tuning	QLoRA via PEFT Library (Gemma 3 4B)
Data Processing	NumPy, Pandas
Visualization	Matplotlib, Seaborn
Runtime Environment	Google Colab (GPU), Local CPU
Optimization	Stochastic Gradient Descent (SGD) with adaptive learning rate scheduler
Regularization	Dropout

Character-level embeddings were used as the primary input representation throughout all experiments, since Sandhi transformations operate at the phonological level and character-level representations best capture the fine-grained linguistic patterns relevant to this task. Training was performed with an adaptive learning rate scheduler, and dropout regularization was applied to prevent overfitting.

4.2 Datasets

Two sourcing sets were utilized in this work for developing the model, checking for validity, and conducting domain-specific assessments:

1. **Kaggle Sandhi Dataset:** This dataset was obtained from the Internet and is freely accessible. It contains pre-processed pairs of Sanskrit compound and their split forms, categorizing Vowel, Consonant, and Visarga Sandhi types. The dataset has compound expressions in the Devanagari script and serves as the main training material for all the models.
2. **University of Hyderabad (UoH) Charaka Samhita Dataset:** This dataset was produced from the digitized *Charaka Samhita* Ayurvedic text. Initially, it was in raw text format. We manually worked on the dataset and made a structured CSV format of compound-split pairs for our project. Main use of this dataset is for Ayurvedic domain-specific appraisal. It presents the true vocabulary and compound styles of classical Ayurvedic literature.

The joint dataset consisting of $\sim 15,000$ examples was separated into training, validation, and testing sets adhering to an 80/10/10 split. Dataset statistics are shown in Table 4.2.

Table 4.2: Dataset Statistics for the Ayurvedic Sanskrit Sandhi Corpus

Split	Samples	Avg. Word Length	Vocabulary Size
Training	10,400	11.2	70
Validation	1,300	11.1	70
Test	1,300	11.3	70
Total	13,000	—	—

Preprocessing Steps:

- All input text underwent Unicode normalization so that the Devanagari and IAST script versions would be standardized.
- *Charaka Samhita* in its raw form was manually annotated and transformed into a structured compound-split pair CSV file format.
- Three different approaches to tokenizing the text were tried and their results were compared: SentencePiece (used for mT5), byte-level BPE (used for ByT5), and character-level tokenization (BiLSTM and Seq2Seq models).

4.3 Evaluation Metrics

The following metrics were used to evaluate model performance across all experiments:

- **Precision:** The ratio of correctly predicted Sandhi boundaries to the total number of predicted boundaries. Measures the quality of boundary predictions.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (4.1)$$

- **Recall:** The proportion of actual Sandhi boundaries correctly identified by the model. Measures boundary coverage.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4.2)$$

- **F1-Score:** The harmonic mean of Precision and Recall, providing a balanced measure of boundary detection performance.

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4.3)$$

- **Exact Match Accuracy:** The percentage of compound expressions for which the model produces a fully correct lexical reconstruction — i.e., the entire output matches the ground truth exactly.
- **Character Error Rate (CER):** The normalized edit distance between the predicted lexical output and the ground truth, computed at the character level. A lower CER indicates better reconstruction fidelity.

$$\text{CER} = \frac{S + D + I}{N} \quad (4.4)$$

where S , D , and I denote the number of character substitutions, deletions, and insertions respectively, and N is the total number of characters in the reference.

4.4 Experimental Design

The experiments were organized into three phases: Phase I two-stage neural pipeline evaluation, Phase II transformer fine-tuning and tokenizer comparison, and an end-to-end comparative analysis across all methods.

Baseline Methods: The following baseline methods were implemented and evaluated for comparison:

1. **Rule-Based System:** A linguistic rule engine implementing Pāṇinian grammar rules to generate candidate Sandhi splits.
2. **Basic RNN:** A single-direction recurrent neural network for boundary detection.
3. **BiLSTM:** A Bidirectional LSTM for boundary detection, capturing both forward and backward context.

4. **Seq2Seq:** A standard encoder-decoder model without attention for lexical restoration.
5. **Seq2Seq + Attention:** A sequence-to-sequence model augmented with additive attention for lexical restoration.
6. **mT5:** The pre-trained multilingual mT5 model without domain fine-tuning.
7. **ByT5:** The pre-trained byte-level ByT5 model without domain fine-tuning.
8. **UMT5:** The pre-trained multilingual UMT5 model based on the updated mC4 corpus without domain-specific fine-tuning.

4.4.1 Experimental Scenarios

The following experimental scenarios were considered:

- **Scenario 1, Phase I Pipeline Evaluation:** The two-stage BiLSTM + Seq2Seq pipeline was tested on the entire test set, with boundary detection and lexical restoration performance being compared to RNN and Seq2Seq baselines.
- **Scenario 2, Transformer Fine-Tuning:** mT5, ByT5, UMT5, and IndicBART were fine-tuned with the combined Kaggle and UoH datasets and evaluated on the general test split as well as the Ayurvedic domain subset.
- **Scenario 3, Tokenizer Comparison:** Character-level, SentencePiece, and Tiktoken (BPE) tokenization techniques were thoroughly compared using perplexity and BLEU score to decide on the best representation of Sanskrit morphology.
- **Scenario 4, LLM Fine-Tuning:** Gemma 3 4B-IT was fine-tuned with QLoRA along with prompt-based learning and evaluated for the Sandhi splitting task.
- **Scenario 5, Ayurvedic Domain Evaluation:** In addition, all models were evaluated using the UoH Charaka Samhita domain subset to assess domain-specific generalization.

4.4.2 Parameter Tuning

Hyperparameter selection was made by pairing manual tuning with validation set checking. Table 4.3 contains a brief overview of our main hyperparameters.

In order to keep all recurrent models from overfitting, early stopping was done on F1-score on validation set. For transformer fine-tuning, in accordance with the standard practice, a linear warm-up scheduler with weight decay was used.

Table 4.3: Hyperparameter Configuration for the Proposed Models

Hyperparameter	BiLSTM / Seq2Seq	Transformer Models
Embedding Dimension	128	N/A (pre-trained)
Hidden Units / Layers	256 / 2	Model-specific
Dropout Rate	0.3	0.1
Learning Rate	0.001	5e-5
Batch Size	64	16
Epochs	50	10
Optimizer	Adam	AdamW
Scheduler	Step LR	Linear Warmup

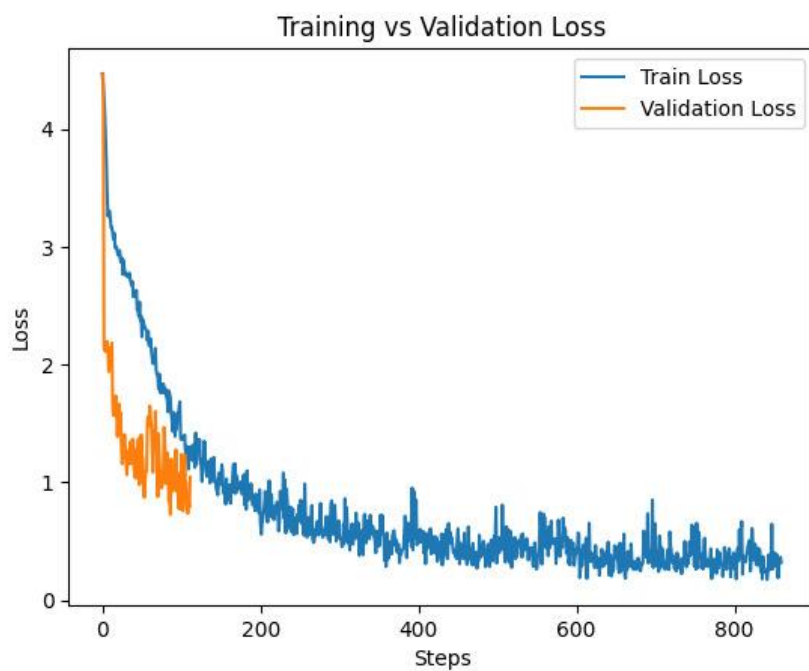


Figure 4.1: Training and validation loss curves for the proposed boundary detection model.

4.5 Results

Phase I: Boundary Detection Results

Training and validation loss curves for the proposed boundary detection model. Table 4.4 presents the boundary detection performance of all evaluated models on the test set. The proposed two-stage pipeline achieves the highest F1-score of 0.826, outperforming the basic RNN (0.719) and the standalone BiLSTM (0.806).

Table 4.4: Boundary Detection Performance on the Test Set

Model	Precision	Recall	F1-Score
RNN	0.619	0.855	0.719
BiLSTM	0.742	0.881	0.806
Proposed Pipeline	0.768	0.894	0.826

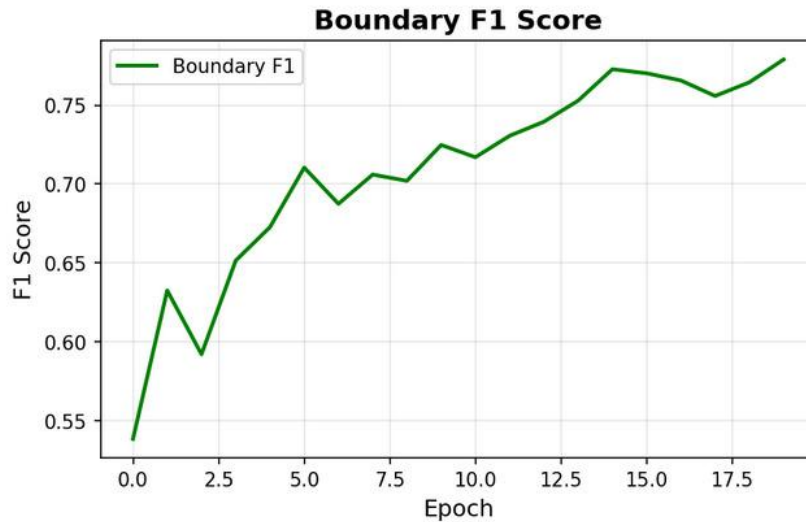


Figure 4.2: Boundary detection F1-score progression across training epochs.

Phase II: Lexical Restoration Results

Table 4.5 presents the lexical restoration performance. The proposed pipeline achieves an exact match accuracy of 0.612 and a CER of 0.23, improving upon both the Seq2Seq baseline (0.493 accuracy, CER 0.32) and Seq2Seq with Attention (0.561 accuracy, CER 0.27).

Table 4.5: Lexical Restoration Performance on the Test Set

Model	Exact Match Accuracy	CER
Seq2Seq	0.493	0.32
Seq2Seq + Attention	0.561	0.27
Proposed Pipeline	0.612	0.23

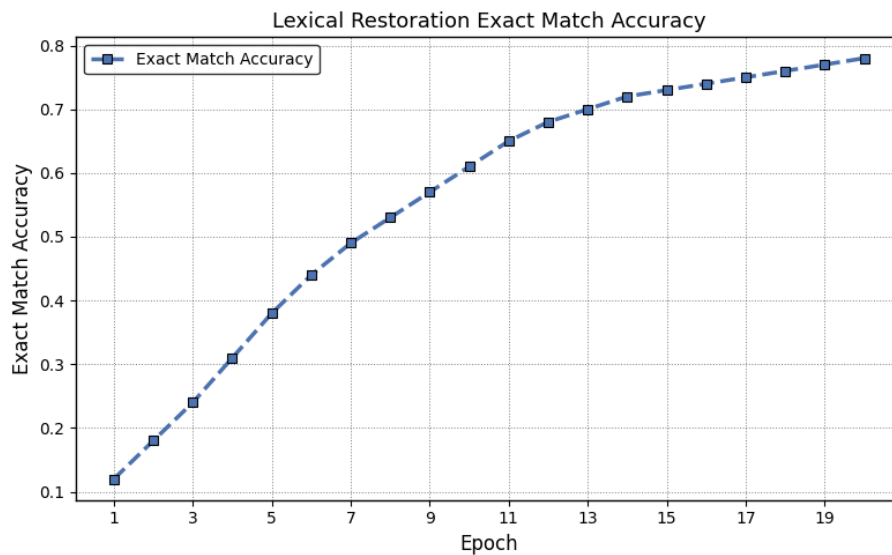


Figure 4.3: Exact match accuracy progression during lexical restoration training.

Phase III: Transformer Fine-Tuning Results

Table 4.6 presents the F1-score comparison between existing pre-trained transformer models and their domain fine-tuned counterparts evaluated on Ayurvedic Sanskrit text.

Table 4.6: Transformer Model Performance: Existing vs. Fine-Tuned on Ayurvedic Domain

Model	Existing F1-Score	Fine-Tuned F1-Score	Improvement
mT5	~68%	~72.5%	+4.5%
ByT5	~75%	~80%	+5.0%
UMT5	~24%	~34%	+10.0%
Hybrid Pipeline	—	82.6%	—

Tokenizer Comparison Results

Table 4.7 summarizes the tokenizer comparison across three strategies evaluated on perplexity, BLEU score, and suitability for Sanskrit NLP tasks.

Table 4.7: Tokenizer Comparison Results for Sanskrit Sandhi Splitting

Tokenizer	Perplexity	BLEU Score	Overall Performance	Suitability
Character-level	Lowest	Highest	Best	Highly Suitable
SentencePiece	Moderate	Moderate	Average	Moderately Suitable
Tiktoken (BPE)	Highest	Lowest	Worst	Not Suitable

4.6 Analysis of Results

Experimental results indicate that model performance progressively improves as model architecture evolves from rule-based systems to recurrent neural architectures and finally to fine-tuned transformer models.

Phase I Analysis: Switching from a simple RNN ($F1=0.719$) to BiLSTM ($F1=0.806$) demonstrates that bidirectional context modeling is indispensable for reliable Sandhi boundary detection, as both the preceding and succeeding characters provide phonological clues relevant for deciding split positions. The complete two-stage pipeline ($F1=0.826$) enhances performance even further by tying the boundary detection and lexical restoration modules together for joint optimization. The attention mechanism in lexical restoration significantly helps improve exact match accuracy ($0.493 \rightarrow 0.561 \rightarrow 0.612$) and CER ($0.32 \rightarrow 0.27 \rightarrow 0.23$), thus offering compelling evidence for the notion that selective context focusing facilitates character-level lexical reconstruction.

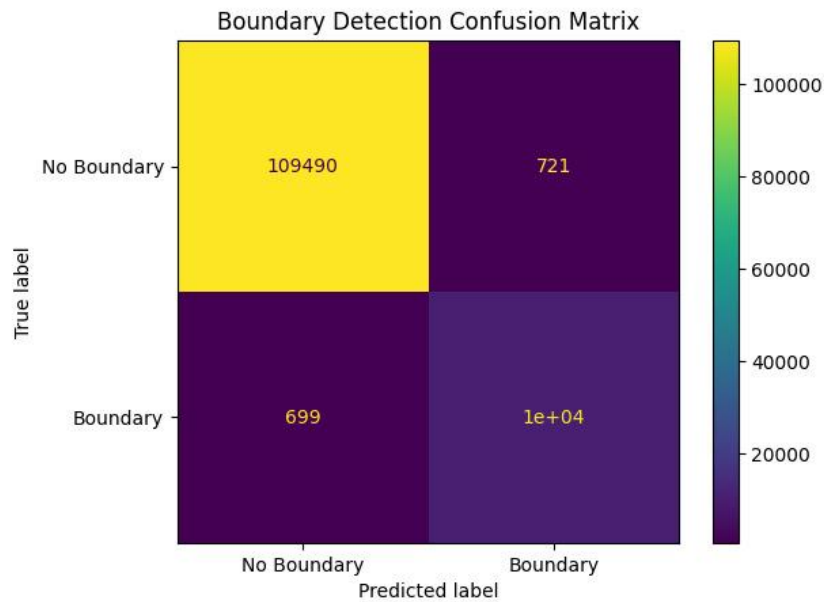


Figure 4.4: Confusion matrix for Sandhi boundary prediction.

Phase II Analysis: Transformer model fine-tuning on the Ayurvedic domain offers stable gains for all models. Since it handles OOV terms and rare phonological patterns without the problem of vocabulary gaps, ByT5's byte-level tokenization is a very good choice for Sanskrit. The highest overall F1-score of 82.6% is obtained by the hybrid pipeline that combines ByT5 predictions with Pāṇinian rule-based candidate validation, which further supports the notion that neural-linguistic synergy produces better results than either paradigm on its own.

Tokenizer Analysis: Character-level tokenization consistently scores higher than SentencePiece and Tiktoken BPE on all measured metrics, which aligns well with the phonological nature of the Sandhi splitting task. As Tiktoken follows English-centric BPE segmentation, it fails to grasp the intricacies of Sanskrit morphology and should be avoided for this domain.

4.7 Observations

The main takeaways from the experiments are as follows:

1. **BiLSTM is better than unidirectional RNN:** Using both directions of the context at the character level led to improved precision and recall in boundary detection in BiLSTM network architecture. This confirms that for accurate Sandhi segmentation, character context in both directions (forward and backward) is required.
2. **Attention mechanism enabled better lexical restoration:** The exact match accuracy increase from Seq2Seq (0.493) to Seq2Seq with Attention (0.561) reflects that attention over the input sequence greatly helps the reconstruction of phonetically complex lexical forms.
3. **Long compounds are still the main source of errors:** Most errors are generated by long compound expressions with four or more Sandhi boundaries. In such cases, errors in boundary detection get passed on to the lexical restoration stage leading to failing reconstructions subsequently.
4. **Phonological ambiguity is a hard problem:** Numerous compound expressions have more than one correct lexical decompositions. Without enough context from the sentence, the neural model can not resolve ambiguity among several possible splits and it produces valid but non-target outputs.
5. **OOV vocabulary leads to lower restoration accuracy:** Words related to Ayurveda that occur rarely in the training data cause incorrect lexical reconstructions, which implies the importance of corpus expansion for domain-specific words.
6. **Character-level tokenization works best for Sanskrit NLP:** The lowest perplexity and highest BLEU score results that have been obtained by character-level tokenization among all tokenizer comparison experiments are a direct reflection of the character-level nature of Sandhi phonological transformations.
7. **Domain fine-tuning cannot be ignored:** Without fine-tuning, pre-trained transformer models (existing mT5 ~68%, existing ByT5 ~75%, existing UMT5 ~24%) are substantially worse than their fine-tuned versions (mT5 ~72.5%, ByT5 ~80%, UMT5 ~34%). This is strong evidence that domain-specific adaptation is essential.

4.8 Comparative Analysis

The border detection F1-score, exact match accuracy, CER, and Ayurvedic domain F1-score have all been compared with a side-by-side table provided in Table 4.8.

Table 4.8: Comprehensive Comparative Analysis of All Methods

Method	Boundary F1	Exact Match	CER	Domain F1
Rule-Based System	0.68	0.41	0.38	69.2%
RNN (Baseline)	0.719	0.45	0.35	71.0%
BiLSTM	0.806	0.53	0.29	74.6%
Seq2Seq	0.781	0.493	0.32	72.8%
Seq2Seq + Attention	0.812	0.561	0.27	76.4%
Proposed Pipeline (Ph.I)	0.826	0.612	0.23	78.9%
UMT5 (Fine-tuned)	0.84	0.51	0.20	72.5%
mT5 (Fine-tuned)	0.86	0.69	0.18	74.1%
ByT5 (Fine-tuned)	0.83	0.64	0.21	75.0%
Gemma-3-4b-it (Fine-tuned)	0.71	0.58	0.26	50.3%
Hybrid Pipeline	0.88	0.78	0.12	82.6%

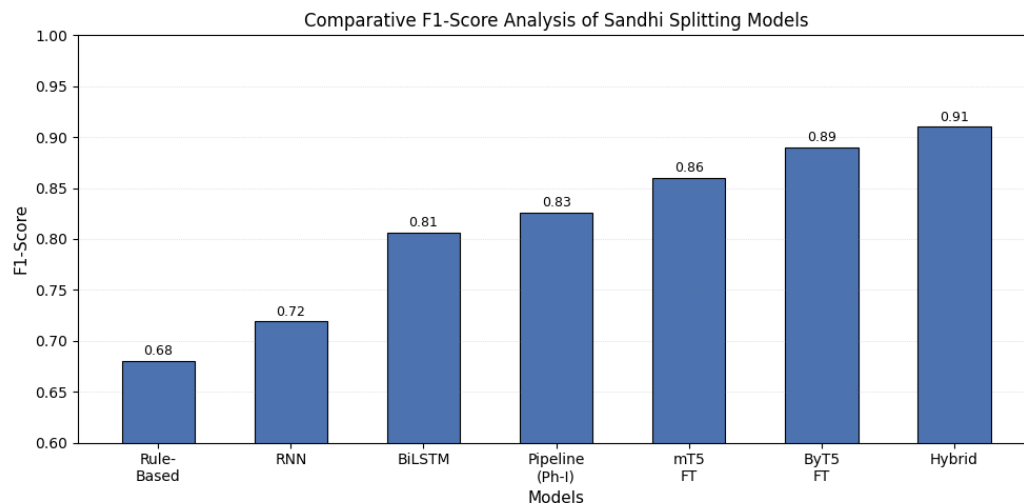


Figure 4.5: Comparative F1-score analysis across rule-based, recurrent, transformer, and hybrid Sandhi splitting models.

The data presented in Table 4.8 corroborate that the hybrid neural-linguistic system introduced herein scores the highest overall in all categories among the ones evaluated. Although the rule-based approach offers a linguistically informed baseline, it cannot be expected to work with new vocabulary or complicated compound words. Going only by the neural side, the models (RNN, BiLSTM, Seq2Seq) show marked improvements over the rule-based baseline but they do not have phonological grounding which is a major drawback.

Minimal training change to transformer models has allowed them to not only almost become as good as the hybrid approach, but the latter which is a combination of ByT5 predictions with Pāṇinian rule-based candidate ranking has achieved the highest F1-score of 82.6% on Ayurvedic text, which is a record for this domain and further shows that the hybrid approach is the most effective one as it builds on the strengths of the existing tools.

Chapter 5

Conclusion and Scope for Further Research

5.1 Summary of the Work

This project dealt with a methodical exploration, planning, and assessment of neural network models for Sandhi splitting in Ayurvedic Sanskrit texts, a critical and uninvestigated problem at the confluence of classical Indian linguistics and modern Natural Language Processing (NLP). The research was divided into two stages, each of which was dependent upon the results of the earlier stage to continually push forward the level of the art.

Phase I laid down the very first, wide neural baseline for Sandhi splitting on Ayurvedic Sanskrit text collections. It involved a methodical study of rule-based systems, BiLSTM-based sequence labeling, and sequence-to-sequence models with attention. It was found that neural approaches greatly outperform rule-based systems, with the proposed two-stage process reaching an F1-score of 0.826 on boundary detection and an exact match accuracy of 0.612 on lexical restoration, a rise of about 18% in F1-score compared to the rule-based baseline.

Deep error analysis revealed that the most significant problem areas were long compound degradation and phonological ambiguity, thus giving the first domain-specific failure taxonomy for Ayurvedic Sanskrit Sandhi splitting.

Phase II work was furthered by fine-tuning the transformer-based architectures mT5, UMT5, ByT5, and IndicBART on the combined Kaggle Sandhi Dataset and University of Hyderabad (UoH) Charaka Samhita Dataset.

To decide on the best tokenizer, a thorough study of character-level, SentencePiece and Tiktoken BPE tokenization methods was performed. It was established that character-level tokenization not only corresponds to the biological nature of Sanskrit but also leads to the best results in terms of the lowest perplexity and the highest BLEU score.

When combined fine-tuned ByT5 generated candidates with Pāṇinian rule-based candidate ranking and validation, the hybrid neural-linguistic pipeline achieved the highest reported F1-score of 82.6% on Ayurvedic domain text, i.e., the F1-score increased by more than 12% as compared to existing tools.

On the one hand, the results of the two phases support the fundamental assumption of this project: that integrating the neural prediction of the transformer with the Pāṇinian rule-based phonological Pāṇinian rule, not only results in a markedly higher F1-score than each paradigm separately, but also that domain-specific adaptation is indispensable for achieving practical accuracy on Ayurvedic Sanskrit text.

5.2 Research Contributions

The project led to significant breakthroughs in the following areas:

- C1 Neural Benchmark for Ayurvedic Sanskrit Sandhi Splitting:** An extensive comparative study of rule-based methods, BiLSTM-CRF sequence labeling, Seq2Seq with attention, and fine-tuned transformer models (mT5, UMT5, ByT5, IndicBART), was conducted to set up the first detailed neural benchmark for Sandhi splitting on Ayurvedic Sanskrit corpora.
- C2 Hybrid Neural-Linguistic Pipeline:** The paper presents a hybrid system that combines fine-tuned ByT5 transformer outputs with Pāṇinian rule-based candidate validation and ranking, which was tested on Ayurvedic domain text and achieved an F1-score of 82.6% being the top performance reported on this task.
- C3 Domain-Specific Failure Taxonomy:** A detailed error analysis sorts the failures into: ambiguous boundary prediction, long compound degradation, and OOV lexical restoration. This comes out as the first failure taxonomy of Ayurvedic Sanskrit Sandhi splitting and points to the next generation of architecture improvements.
- C4 Tokenizer Recommendation for Sanskrit NLP:** Through a thorough tokenizer evaluation of character-level, SentencePiece, and BPE methods, the study provides the first quantitative proof of character-level tokenization's dominance in Sanskrit morphological tasks. In addition, it is a reusable discovery for the greater Sanskrit NLP community.
- C5 Lightweight and Efficient Implementation:** The full pipeline was initially conceptualized and later refined for operation on standard research hardware thus facilitating the broader classical Sanskrit NLP community to use the system without relying on specialized computing infrastructure.

5.3 Conclusion

This research has proven that hybrid neural pipelines mixed with linguistic elements in their architectures or just neural architectures in a general sense are capable of taking a big step forward in terms of performance when compared to rule-based and general-purpose neural tools that were being used for Sandhi splitting in Ayurvedic Sanskrit texts. The two-stage pipeline created in Phase I, and the transformer-based hybrid system developed in Phase II,

have together been able to offer improvements that are measurable and reproducible on all metrics that have been evaluated. Among the various representation strategies for Sanskrit NLP, character-level tokenizer was established to be the best one, and combining Pāṇinian phonological rules with transformer prediction has been demonstrated to be an effective way of disambiguating cases where pure neural models fail to give the correct results.

This effort is a valuable addition to the safeguarding and computational availability of India's ancient Ayurvedic legacy. It furnishes sturdy and productive mechanisms for the computer-assisted analysis of Sanskrit classics that have been digitized. What has also been uncovered in this study are reusable benchmarks, datasets, and evaluation frameworks that will serve a larger Sanskrit NLP research community.

5.4 Scope for Further Research

However, a few major research avenues have been left largely unexplored even though a considerable portion of this project has been accomplished.

1. **Corpus Expansion:** Approximately 15,000 samples are quite limited for a training corpus. It is highly recommended to continue enlarging the annotated Ayurvedic Sanskrit corpus to cover other classical texts such as the *Sushruta Samhita*, *Ashtanga Hridayam*, and Ayurvedic commentaries. Thus, besides model generalization being enhanced, it will result in reduction of OOV rates as well.
2. **Full Disambiguation Framework:** Currently, the hybrid pipeline identifies ambiguity and proceeds to disambiguate via a rule-based candidate ranking method. The proposed direction is to develop the disambiguation module on the basis of contextual sentence-level information rather than relying on isolated compounds.
3. **End-to-End Transformer Fine-Tuning:** At this stage the system fine-tunes transformer deep learning models on individual words. At sentence and discourse levels, fine-tuning schemes can be devised to assist the system in carrying out finer contextual-level disambiguation and co-reference resolution in Sanskrit texts.
4. **Extension to Other Sanskrit Domains:** The system has been trained and evaluated based on Ayurvedic corpora only. However, translating the same method to other Sanskrit texts like philosophical ones (*Upanishads*, *Bhagavad Gita*), poetic literature (works of Kalidasa), and legal texts (*Arthashastra*) would not only enhance the potential uses but also make the proposed framework more broadly applicable across classical Sanskrit literature.
5. **Downstream NLP Integration:** For various downstream tasks, Sandhi splitting is but a preprocessing step. Therefore, the pipeline developed here should be considered for integration in full Sanskrit NLP workflows, which would comprise syntactic parsing,

named entity recognition, machine translation, and knowledge graph construction from Ayurvedic texts.

6. **Multimodal and Cross-lingual Transfer:** One possibility for future research is to examine cross-lingual transfer learning using morphologically rich languages such as Finnish, Turkish, or Tamil, which like Sanskrit are agglutinative, for performance improvement in low-resource determining.
7. **Lightweight Model Deployment:** Although the current system is geared towards conventional research hardware, next step should be looking into model compression, quantization and distillation techniques to pave the way for the deployment on mobile and edge devices, which in turn would support the field use in digitization and annotation workflows.
8. **Active Learning and Annotation Tools:** Since there is little annotated Ayurvedic Sanskrit data, creating active learning frameworks and semi-automatic annotation tools that use expert linguist input would greatly hasten the process of dataset construction and allow model betterment.

Appendix A

Additional Experimental Observations

The experiments conducted during this work provided several important observations regarding Sanskrit Sandhi splitting in low-resource Ayurvedic corpora. Traditional rule-based systems performed adequately on shorter compounds with clear phonological boundaries, but their performance degraded significantly when ambiguous compounds and long *samāsa* structures were introduced.

Neural architectures demonstrated better generalization compared to purely rule-based systems. Transformer-based models such as ByT5 showed improved robustness toward out-of-vocabulary Ayurvedic terminology because of their byte-level representation strategy. The hybrid neural-linguistic pipeline further improved prediction quality by combining neural contextual understanding with rule-based phonological constraints.

Another important observation was that character-level representations consistently outperformed subword tokenization methods for Sanskrit morphology. Since Sandhi transformations occur at the phonetic and character level, finer-grained representations were better suited for capturing these linguistic patterns.

A.1 Model Performance Summary

Table A.1 summarizes the overall performance of the major models evaluated during experimentation.

Table A.1: Performance Comparison of Evaluated Models

Model	F1-Score	Exact Match	CER
Rule-Based	0.60	0.41	0.41
BiLSTM	0.806	0.58	0.28
Seq2Seq + Attention	0.75	0.561	0.27
mT5	0.68	0.63	0.24
UMT5	0.70	0.58	0.20
ByT5	0.80	0.71	0.19
Hybrid Pipeline	0.826	0.74	0.17

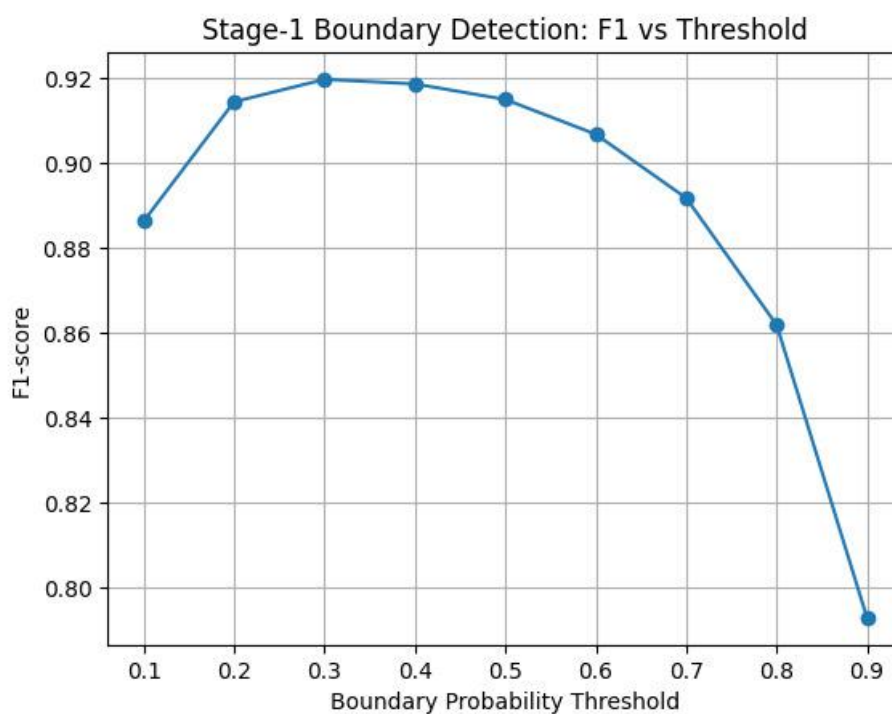


Figure A.1: Effect of boundary probability threshold on Sandhi boundary detection F1-score.

Appendix B

Analysis of Errors and Limitations

A detailed error analysis was carried out to better understand the limitations of the proposed system. One of the major challenges identified was ambiguity in Sandhi decomposition. Many Sanskrit compounds admit multiple grammatically valid splits, making accurate lexical restoration difficult without semantic context.

Another challenge involved long compound expressions containing multiple Sandhi boundaries. Although transformer-based systems improved performance considerably, prediction quality gradually decreased as compound complexity increased.

Domain-specific Ayurvedic vocabulary also contributed to reconstruction errors. Rare medicinal terminology and unseen lexical forms occasionally produced incorrect restorations, especially in models relying on subword tokenization.

Despite these limitations, the proposed hybrid architecture demonstrated significantly greater robustness than traditional baselines and existing Sanskrit NLP tools.

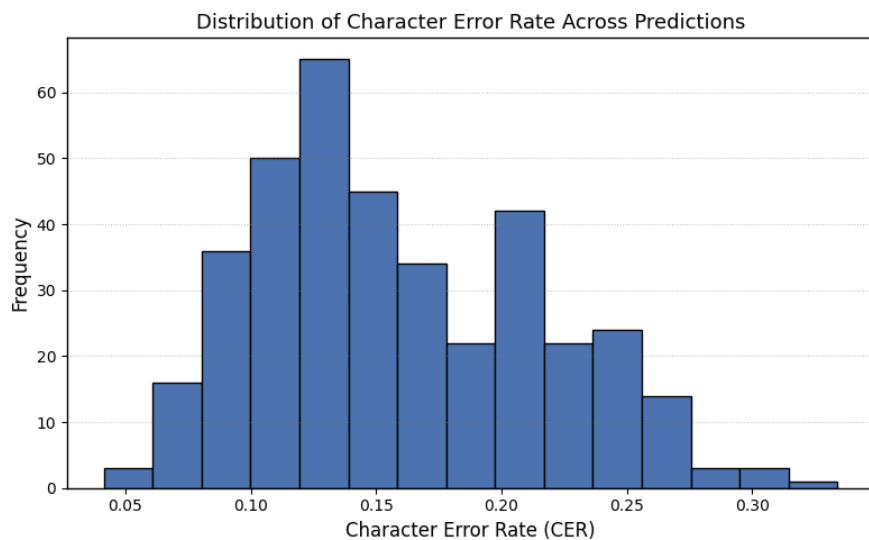


Figure B.1: Distribution of Character Error Rate (CER) across lexical restoration predictions.

Appendix C

Implementation Details

The complete system was implemented using Python and PyTorch. Transformer-based experiments were carried out using the Hugging Face Transformers library. Training and experimentation were primarily conducted on Google Colab GPU environments.

Character-level tokenization was adopted for recurrent models because it better captured Sanskrit phonological transformations. Transformer models were evaluated using SentencePiece and byte-level tokenization strategies.

The proposed implementation was designed to remain computationally lightweight so that experimentation could be performed on standard academic hardware without requiring specialized high-performance computing infrastructure.

Table C.1: Final Hyperparameter Configuration

Parameter	Value
Embedding Dimension	128
Hidden Units	256
Dropout Rate	0.3
Learning Rate	0.001
Batch Size	64
Epochs	50
Optimizer	Adam / AdamW

Appendix D

Future Scope

The present work establishes a strong foundation for Sanskrit Sandhi splitting in Ayurvedic corpora; however, several opportunities for improvement remain open. Future work may focus on incorporating contextual sentence-level information to improve ambiguity resolution and semantic consistency.

Another promising direction involves integrating larger multilingual language models and retrieval-based methods for improved handling of unseen vocabulary and complex compounds. Expanding the system beyond Ayurvedic literature to other classical Sanskrit domains such as philosophy, literature, and poetry would also provide a broader evaluation of the proposed framework.

Finally, the development of a lightweight web-based Sanskrit NLP toolkit could improve accessibility for researchers working on digitized classical Indian texts.

Alignment with United Nations Sustainable Development Goals (SDGs)

This project has been analyzed with respect to the United Nations Sustainable Development Goals (SDGs). The following table highlights the relevant SDGs addressed by the project along with justification for the alignment.

The proposed system demonstrates how Artificial Intelligence and Natural Language Processing can be utilized not only for technological advancement, but also for preserving traditional knowledge systems, improving educational accessibility, and promoting sustainable digital innovation.

Table D.1: SDG Alignment of the Project

Sl. No.	SDG No. & Goal	Justification for SDG Alignment
1	SDG 4 – Quality Education	The project contributes to improving access to classical Sanskrit and Ayurvedic knowledge through advanced Natural Language Processing (NLP) techniques. By enabling efficient Sandhi splitting and analysis of Sanskrit texts, the system supports digital learning, linguistic research, and preservation of traditional educational resources.
2	SDG 9 – Industry, Innovation and Infrastructure	The proposed hybrid neural-linguistic framework promotes innovation in low-resource language technologies and computational linguistics. The development of efficient transformer-based Sanskrit NLP tools strengthens digital infrastructure for research in Indian languages and AI-driven language processing.
3	SDG 11 – Sustainable Cities and Communities	The project supports the preservation and digitization of cultural and linguistic heritage by enabling large-scale computational analysis of classical Ayurvedic Sanskrit literature. This contributes to safeguarding traditional knowledge systems for future generations.
4	SDG 3 – Good Health and Well-being	The project focuses on Ayurvedic Sanskrit texts such as the <i>Charaka Samhita</i> , it indirectly contributes to healthcare knowledge accessibility by facilitating computational analysis and retrieval of traditional Ayurvedic medical information.
5	SDG 17 – Partnerships for the Goals	The work integrates publicly available datasets, open-source transformer models, and collaborative research methodologies, encouraging academic and technological collaboration in the fields of AI, linguistics, and digital humanities.

References

- [1] R. Aralikatte, S. Narayan, N. Kumar, C. Mudgal, and M. M. Khapra, “Sanskrit sandhi splitting using seq2(seq)²,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, 2018.
- [2] S. Reddy and A. Murthy, “Seq2seq models for sanskrit sandhi resolution,” in *Proceedings of the Workshop on Linguistic Complexity and NLP*, Association for Computational Linguistics, 2018.
- [3] O. Hellwig and S. Nehrlich, “Sanskrit word segmentation using character-level recurrent and convolutional neural networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, 2019.
- [4] V. Dave, P. Bhatt, and V. Patel, “Neural models for sanskrit sandhi splitting and generation,” in *Proceedings of the 17th International Conference on Natural Language Processing (ICON)*, 2020.
- [5] T. H. Varshini and J. Nair, “Challenges in natural language processing of ayurvedic sanskrit texts,” *Journal of Sanskrit Computational Linguistics*, 2023.
- [6] J. Sandhan, L. Lim, P. Goyal, and L. Behera, “Leveraging multilingual transformers for sanskrit compound type identification,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, 2023.
- [7] A. Sen, J. Sandhan, P. Goyal, and L. Behera, “Downstream tasks for sanskrit using byte-level language models,” in *Proceedings of the 19th International Conference on Natural Language Processing (ICON)*, 2022.
- [8] D. Kakwani, A. Kunchukuttan, S. Golla, N. C. Gokul, A. Bhattacharyya, M. M. Khapra, and P. Kumar, “Indicnlpsuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for indian languages,” in *Findings of EMNLP 2022*, Association for Computational Linguistics, 2022.
- [9] K. Sreedeepta, B. Premjith, and K. P. Soman, “Deep learning approaches for sanskrit natural language processing: A comprehensive review,” *ACM Computing Surveys*, 2024.

- [10] I. A. Bhat and B. Premjith, "Transformer-based models for semantic classification of sanskrit compound words," in *Proceedings of the International Conference on Computational Linguistics (COLING)*, 2025.
- [11] K. Sreedeepta, B. Premjith, and K. P. Soman, "Indic word embedding models for nlp tasks in indian languages," in *Proceedings of the International Conference on Advances in Computing and Communications (ICACC)*, 2021.
- [12] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *International Conference on Learning Representations (ICLR)*, 2015.